

Intelligent Candidate Discovery & Ranking

A hybrid AI recruiter that ranks candidates for Redrob's Senior AI Engineer role by understanding career context, not keyword density.

Problem

- Recruiters miss strong fits because keyword filters reward profile stuffing.
- The JD explicitly warns that Marketing Managers with AI skills are traps.
- Behavioral signals (availability, response rate) matter as much as skills.
- Ground truth scoring weights top-10 quality heavily ($\text{NDCG@10} = 50\%$).

Design Principles

- Read the JD as requirements + anti-patterns, not a bag of words.
- Separate offline precompute from the 5-minute CPU-only ranking step.
- Every reasoning string must cite facts present in the candidate profile.
- Penalize honeypots and title/skill inconsistency before semantic ranking.

Architecture

- Phase A (precompute): stream 100K JSONL, disqualify honeypots, embed profiles with TF-IDF+SVD.
- Phase B (rank): cosine similarity + weighted structured sub-scores.
- Output: validated CSV with monotonic scores and evidence-backed reasoning.

Scoring Formula

- Final score = penalty x (0.35 technical + 0.25 career + 0.20 availability + 0.12 seniority + 0.08 semantic).
- Technical fit tracks must-haves: embeddings/retrieval, vector DB, Python, eval metrics.
- Career quality rewards product-company ML roles, stable tenure, ranking/search delivery.
- Availability uses open-to-work, recency, recruiter response, notice period.

Trap & Honeytrap Handling

- Hard disqualify: impossible YoE vs career timeline, expert skills with zero usage.
- Heavy penalty: non-ML titles with 6+ AI keywords (HR Manager, Accountant, etc.).
- Soft penalty: consulting-only careers, CV-only focus, framework-demo profiles.
- Matches submission spec guidance and JD 'read between the lines' section.

Why This Beats Keyword Matching

- A keyword ranker puts HR Managers with stuffed skills at rank 1 (see sample_submission.csv).
- Our title-career consistency gate surfaces real ML engineers with production retrieval evidence.
- Semantic similarity rescues candidates who describe ranking systems without saying 'RAG'.
- Behavioral down-weighting removes perfect-on-paper but unavailable candidates.

Reproducibility

- `python precompute.py`
- `python rank.py --candidates ../dataset/candidates.jsonl --out output/submission.csv`
- Runs on CPU, no network during ranking, artifacts cached under `artifacts/`.
- Streamlit sandbox (`app.py`) supports small-sample validation for organizers.